WEBER Loïc

DERRIDJ Mellyna



TP 1 et 2 – Partie 1 - Microcontrôleur

Gestion de chaîne de caractères - Norme NMEA

6.3 . Les liaisons séries

Attention, la sonde utilisée a un facteur x10.



Trame reçue sur l'oscilloscope

La ligne de repos vaut 1 (le bit est sur 1).

Un bit dure environ 88 µs.

Le bit de start est un motif qui permet au récepteur de détecter quand la transmission de la trame commence.

En prenant en compte le bit de start et le bit de stop, on a donc 10 bits pour un octet (8 bits).

Le débit d'émission étant 11400 bits par seconde, on a un débit réel d'environ 9120 bits par seconde (11400*(8/10)).

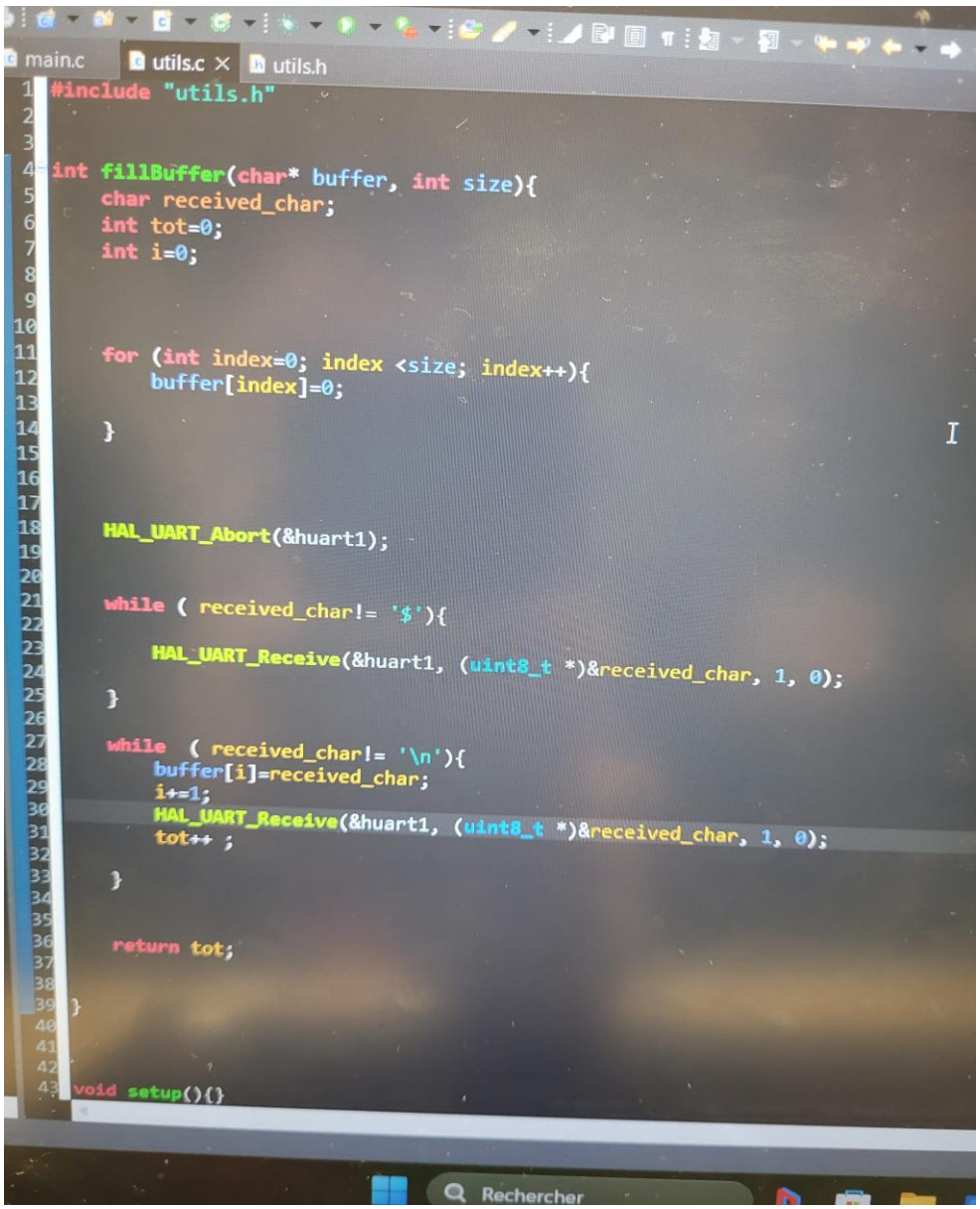
Le premier octet envoyé correspond à 00100100 en binaire et 0x24 en hexadécimal, correspondant tous deux au caractère $ avec la table ASCII.



Image de la trame reçue sur l'oscilloscope sur laquelle on peut notamment voir le premier caractère donné par celle-ci sous forme hexadécimale

Pour la question 6.6.3., nous n'arrivions pas à faire fonctionner notre code malgré qu'il soit quasiment identique à celui d'un groupe pour lequel ça marchait (en sachant que nous avions importé tous les éléments nécessaires dans le main.c et le utils.h) . Notre chargé de TP nous a également aidé en corrigeant notre code mais sans succès. Nous avons ensuite essayé le même code sur une deuxième machine en pensant que le problème pouvait venir de celle que nous utilisions mais ici encore sans succès.

La séance se terminant là-dessus, nous n'avons pas encore pu trouver de solution à ce problème.



A l’issue de ce TP, nous avons tout de même appris à utiliser l’oscilloscope avec une carte nucléo et à s’en servir pour lire les arrivées de bits dans l’oscilloscope.

Quant à la partie sur STM32, nous avons appris le fonctionnement des fonctions HAL_UART_Receive et HAL_UART_Transmit et comment elles travaillent avec les trames données par la nucléo. Pour finir, même si nous n’avons pas pu faire fonctionner la fonction fillBuffer, nous avons compris comment elle fonctionne à chaque étape et ce qu’elle est censée nous retourner.

WEBER Loïc

DERRIDJ Mellyna



TP 1 et 2 – Partie 2 - Microcontrôleur

Gestion de chaîne de

caractères - Norme NMEA

6.6.3

```
int fillBuffer(char* buffer, int size){
    char received_char;
    int tot=0;
    int i=0;



    for (int index=0; index <size; index++){
        buffer[index]=0;
    }



    HAL_UART_Abort(&huart1);


    do {
        HAL_UART_Receive(&huart1, (uint8_t *)&received_char, 1, HAL_MAX_DELAY) ;
    } while ( received_char!= '$');



    do {
        buffer[i]=received_char;
        HAL_UART_Receive(&huart1, (uint8_t *)&received_char, 1, HAL_MAX_DELAY);
        i+=1;
        tot+=1;
    } while  ( received_char!= '\n');

    buffer[i]= received_char;


    return tot;


}
```

Fonction fillBuffer avec un léger délai dans HAL_UART_Receive permettant de récupérer des trames

Dans la fonction fillBuffer, ajouter un délai dans HAL_UART_Receive permet de régler le problème que nous avions à la fin du TP précédent.

La fonction fillBuffer modifie l'index qu'on utilise pour agir sur le tableau mais pas en utilisant un pointeur, donc quand on appellera la fonction, l'index reprendra la valeur qu'il avait avant son passage dans la fonction et pointera donc vers l'indice voulu.

6.7

```
int isGPGGA(char * frame){
    char* debut = "$GPGGA";
    for (int i=0; i<6; i++){
        if ( frame[i]!=debut[i]){

            return -1;
        }
    }

    return 1;

}
```

Fonction isGPGGA qui permet de vérifier si une trame commence par les caractères $GPGGA

```
int extractChecksum(char* buffer){

    while (*buffer!='*' ){
        buffer++;

    }
    if ( buffer[1]>='A' || buffer[2]>='A'){
        return ( (buffer[1] & 0x0F) + 9 * (buffer[1] >= 'A')) * 16
               + ((buffer[2] & 0x0F) + 9 * (buffer[2] >= 'A'));



    }



    return (buffer[1]&0x0f)*16 +(buffer[2]&0x0f);

}




int calculateChecksum(char* buffer){
    int checksum;
    checksum=0;
    int i=1;
    while ( buffer[i]!='*'){
        checksum= checksum ^ buffer[i];
        i++;

    }
    return checksum;
}
```



Fonctions extractChecksum et calculateChecksum

La fonction extractChecksum permet de récupérer la fin d'une trame (précisément tout ce qui se trouve derrière '*').

La fonction calculateChecksum permet de calculer le OU EXCLUSIF deux à deux de tous les éléments précédant '*' dans la trame. Cela permettra ensuite de vérifier si la trame est valide en comparant au résultat de la fonction extractChecksum.

La ligne de code correspondant à la ligne 6 de l'algorithme est celle avec une * jaune dans extractChecksum. Dans cette fonction on a commencé par placé le pointeur de buffer sur '*' qui se trouve en fin de trame. Puis la ligne 6 permet de récupérer les deux valeurs suivant '*'. buffer[index+1]&0x0f permet de récupérer le caractère suivant '*' par exemple, alors que buffer[index+1] seul aurait donné toute la chaîne de caractère suivant '*'. En répétant l'opération deux fois avec les indices qui nous intéressent on peut donc récupérer seuls les deux caractères suivant '*'

```
int intcheckFrame(char* buffer) {
    char fram[7];

    strncpy(fram, buffer, 6);
    fram[6] = '\0';

    if ((isGPGGA(fram) == 1) &&  (calculateChecksum(buffer) == extractChecksum(buffer))) {

        return 1;

        }

    return 0;
}
```

Fonction intcheckFrame permettant de vérifier si une trame est valide

Voici ci-dessous la fonction dans le loop avec les résultats affichés sur la console.

```
char buffer[BUFFER_MAX_SIZE];



void loop(){
    int size = fillBuffer(buffer, BUFFER_MAX_SIZE);

    extractChecksum(buffer);
    printf("Just received %d bytes \r\n",size);
    printf("Checksum -> calculated %x \t extract %x \r\n", calculateChecksum(buffer), extractChecksum(buffer));

    int c=intcheckFrame(buffer);
    printf( " %d \r \n" , c);



}
```

```
Just received 62 bytes
Checksum -> calculated 30        extract 30
0
Just received 63 bytes
Checksum -> calculated 60        extract 60
0
Just received 67 bytes
Checksum -> calculated 74        extract 74
1
Just received 62 bytes
Checksum -> calculated 30        extract 30
0
Just received 63 bytes
Checksum -> calculated 60        extract 60
0
```

```
float getLatitude(char * frame){
    int cpt=0;
    int i=0;
    int lat=0;



    while ( cpt<2){
        i++;
        if (frame[i]==',' ){
            cpt+=1;
        }
    }
    i=i+1;


    lat+=  10* (frame[0+i]&0xf) + (frame[i+1]&0xf) + (1/60 ) * (  (10*frame[i+2]&0xf)  +(frame[i+3]&0xf) +  0.1(frame[i+5]&0xf) + 0.01*(frame[i+6]&0xf)
           + 0.01*(frame[i+7]&0xf));


    int i2=0;
    int cpt2=0;
    while( cpt2<4){
        i2++;
        if (frame[i2]==',' ){
                    cpt2+=1;
                }

    }

    if ( frame[i+9]=='S'){
        return -lat;
    }
    return lat;


}
```

Fonction getLatitude qui permet de récupérer la latitude contenue dans la trame donnée

Malheuresement cette fonction nous retournait une erreur que nous n’avons pas eu le temps de corriger.

Néanmoins, nous avons réussi à corriger le problème que nous avions la dernière fois avec fillBuffer. Nous avons également appris à travailler avec des trames.